SF Data Migration Tool — Selectively Agentic Architecture

1. Division of responsibility

The tool stays a deterministic ETL pipeline: source data is extracted in full to our side, then transformed and loaded into Salesforce. Agents are applied *selectively*, to the two problems that deterministic rules have not solved — getting relationships between objects right, and recovering from load errors. Everything else, including extraction, transformation, loading and ordering, runs exactly as it does today.

Where they are applied, the agents act as a configuration layer: they read system state, decide what the pipeline needs in order to move related data correctly, and then *build that configuration themselves* — which the pipeline executes exactly as it executes hand-written configuration today.

1. **Only the ETL moves data.** Agents read schema, dependency graphs and error summaries. They never write to Salesforce and never modify staged files. The one place an agent sees record values is post-load diagnosis: a capped, truncated sample of the rows Salesforce rejected, because some failures cannot be explained from schema alone. A single setting turns those values off for a client whose data may not leave our side.
2. **Humans own field-to-field mapping.** Choosing that a source field belongs in a particular Salesforce field remains a human decision, supported by a model-assisted pass that proposes pairings and never writes one.
3. **Agents own relationship correctness and error repair, end to end.** Identifier remapping, identifier maps, load ordering, missing parents, and post-load failures. They do not hand a list of problems back to an engineer to wire up — they wire it up, and report what they did. This is the work that is currently manual, iterative, and the main source of failed runs.
4. **Escalate only what cannot be fixed safely.** Anything inside the bounded remedy set is resolved autonomously. A decision that genuinely needs human judgement — a required parent that is out of scope, an ambiguous default — is raised as a blocker instead of guessed at.
5. **What repair cannot fix, a person approves.** Load failures left after automatic repair are explained, and the fix is offered as a proposal with a before/after. Nothing is written until someone clicks Apply, and applying never calls the model.
6. **Every change is validated before it is written.** The agent's work is checked against the live Salesforce describe and an allow-list of transform functions. Anything that fails validation is discarded rather than applied, and everything applied is recorded and reversible.
7. **Staged, not streaming.** All selected source data lands on our side before any transform runs, so repair and retry operate on local data and never re-query the source system.
8. **Degrades to today's behaviour.** If Bedrock is unavailable the agent path is skipped and the pipeline runs unchanged. No model call sits on a data-integrity critical path.

2. Where the agents attach

A migration runs as the numbered sequence below, start to finish, split into the two halves it naturally has: the design-time half on the left, where the migration is configured and nothing has moved yet, and the run itself on the right. Every step already exists today except the highlighted ones — **step 2, step 4 and step 11**, with step 11 now followed by a diagnosis step, **11b** — which are the entire scope of this work. Following the numbers is the clearest way to see how little of the pipeline changes.

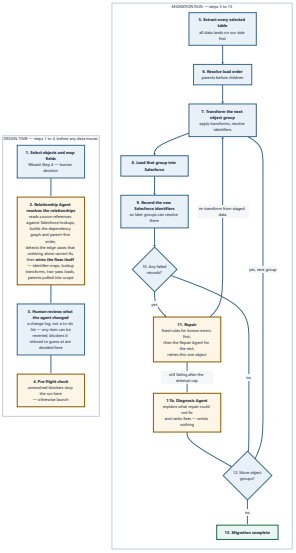


Diagram 1 — The migration procedure end to end. Highlighted steps are the new agent work.

The two halves matter because they carry different risk: everything on the left is reversible at no cost, while everything on the right is touching a live Salesforce org. Note also that step 7 is re-entered from two places: once per object group in the normal loop, and again after a repair — which is why a repair costs one object's transform rather than a restart.

Step 11b is the one step that ends with a person rather than a write. When rows are still rejected after the last repair attempt, the Diagnosis Agent explains why and attaches the mapping changes that would fix it. After the run, the migration engineer can discuss the failure with an assistant, apply one of those fixes as a reviewed proposal, and re-run only the affected object (section 6.4).

Inside step 2 — how the Relationship Agent works

Step 2 is the substance of the design-time half, so it is worth expanding. It proceeds in five passes, each building on the last. The first four work out what is wrong; the fifth is the one that matters operationally, because the agent fixes it rather than reporting it.

1. **Read both sides of the relationship.** On the source side, every field carries a type, and reference fields additionally carry the table they point at — a record's owner field declares that it points at the user table. On the destination side, the Salesforce describe declares which fields are lookups and which objects each lookup accepts. These two facts, put side by side, are what makes dependency discovery possible without reading a single record.
2. **Pair references into dependency edges.** For each source reference field, the agent asks whether the table it points at is also part of this migration, and if so which Salesforce object it becomes. When the answer is yes, that is a dependency edge: the child cannot be loaded correctly until the parent exists. When the answer is no, it is a missing parent — the reference has nothing to resolve against, and it is recorded as a notice whose severity depends on whether the destination field is required.
3. **Derive the load order from the edges.** The set of edges forms a graph, and the graph is sorted so that every parent loads before anything that points at it. This sort is deterministic and stays in code — the agent's job is to make sure the graph is *complete and correct*, particularly that no parent is missing from the selection, because a sort over an incomplete graph produces a confidently wrong order.
4. **Find the edge cases ordering cannot express.** Correct sequencing alone does not make a relationship work. The child mapping must also carry a resolution step that turns the source identifier into the parent's new Salesforce identifier, pointed at the right identifier map — and some shapes cannot be sequenced at all: a record pointing at its own object, two objects pointing at each other, a lookup that accepts several object types, a parent that sits outside the migration's scope. The agent identifies each of these and selects a remedy from the fixed set in section 5.3.
5. **Wire the remedy into the pipeline.** This is the pass that distinguishes the agent from a linter. It does not return a list of things for an engineer to configure — it performs the configuration, then reports what it did. Concretely, it registers the identifier map that the parent mapping will populate during load and attaches the matching lookup transform to the child field, so the child resolves against the parent's new Salesforce identifiers. It pulls a missing parent into the run when the reference cannot work without it. It splits a self-reference or a mutual reference into an insert pass and a follow-up pass that fills the reference once identifiers exist. It writes the resolved parent-first order onto the mappings. It sets a constant or default where a reference points outside scope and a safe default exists. Everything it writes is ordinary pipeline configuration of a kind an engineer would otherwise hand-author, and it is validated against the live Salesforce schema before being saved.

What comes out of step 2 is therefore a **configured migration plus a change log**, not a to-do list. Step 3 is human oversight of work already done: every item can be inspected and reverted individually, and the only things left genuinely outstanding are the blockers the agent deliberately refused to guess at — a

required parent nobody selected, a default with no safe answer. Section 5 covers what it can detect and why that set is deliberately not fixed.

The three new steps in detail

Step	When it runs	Where it lives	What it does
2 — Resolve	On request in wizard Step 4, after the user has made an initial pass at mapping	A new mapping-agent service and endpoint, alongside the existing mappings and compatibility APIs	Registers identifier maps, attaches lookup transforms, pulls in missing parents, sets two-pass handling where needed, and writes the parent-first order — then returns a change log plus any blockers it refused to guess at.
4 — Gate	At launch, before extraction begins	Migration manager, immediately before load-order resolution	Makes no changes. Re-validates the configuration as it now stands and refuses to start on unresolved blockers, reusing the compatibility engine's existing checks.
11 — Repair	Only when an object reports failed records at step 10	Migration manager, inside the per-group loop after the load call returns	Fixed rules first for errors with exactly one correct fix, then the Repair Agent for the rest — field-level patches to transform functions and their parameters, followed by a re-transform and re-load of that single object from staged data. Whatever still fails is diagnosed (11b) and offered to a person as proposed fixes.

> **Why these three positions and no others.** Step 2 is where a mistake costs nothing to fix, because no data has moved. Step 4 is the last moment at which we can still refuse to start. Step 11 is the only point where Salesforce has told us something we could not have known in advance. Anywhere else would place the model between the data and the destination, which the first design rule forbids.

3. How the agent is connected

3.1 Runtime

Each agent is a bounded tool-use loop against the Bedrock Converse API. The model is given a role, a task, and a set of tools; it calls tools to gather facts, then terminates by calling a single submission tool that carries its conclusion. There is no framework in between — the loop is small, and keeping it small is what lets us validate between every turn.

Deployment is a thin service module in the existing FastAPI application. Credentials are resolved server-side and held outside the model's reach: they are captured by the tool executors, never declared in a tool's input schema, so the model can neither see them nor supply them.

3.2 Tools are adapters over logic we already have

This is the reason the agentic layer is a small build rather than a rewrite. The analysis capability already exists in the compatibility engine and is already tested in production runs. The agent does not reimplement any of it — it decides what to inspect and what to conclude.

Tool	Existing capability behind it	What it gives the agent
List source fields	Source provider field discovery	Field names, data types, and which table each reference field points at
List destination fields	Salesforce describe wrapper	Type, length, required, createable, lookup targets, picklist values
Suggest mapping	Deterministic auto-suggest	A name-and-type draft to reason from rather than starting blank
Check type	Type coercion engine	Whether a pair is compatible, and which transform closes the gap
Resolve order	Dependency resolver	Topological order, edges, cycles, and missing parents
Check compatibility	Compatibility report	Current blockers: unset required fields, raw reference copies, picklist gaps
Submit resolution	—	The single, schema-constrained channel through which the agent's decisions leave the loop and enter validation

> **One prerequisite refactor.** The dependency resolver and the compatibility report currently load mapping configurations from the database. The agent reasons over drafts before it commits anything, so both need to accept in-memory configurations. Without this, working out what to change would require writing speculative state first — which also breaks preview mode, where nothing may be written at all.

3.3 Guardrails

Because the agent applies its own conclusions rather than handing them to a person, the guardrails carry more weight than they would in an advisory design. They are what make autonomy safe: the agent can

decide freely, but only within a space where every possible decision is already known to be executable.

- **Fixed transform allow-list.** The agent may only configure generic transform functions already registered in the pipeline. Any other function name is rejected — it cannot introduce new executable behaviour, only new configuration of behaviour that already exists and is already tested.
- **Schema verification before write.** Every destination field it touches is re-checked against a fresh Salesforce describe. Unknown or non-createable fields are dropped rather than written.
- **Ordering is recomputed, never trusted.** Whatever sequence the agent arrives at is discarded, and the deterministic resolver is re-run over the configuration it produced. Its real contribution is making the graph complete — ensuring the parent is *in* the selection and wired — while the machine decides the order.
- **Bounded execution.** Turn cap, token cap, wall-clock cap per session, a capped number of repair attempts per object, and a global feature flag.
- **Everything applied is reversible.** Each change is recorded against the session and object that produced it, so a review can revert one item without unpicking the rest.
- **Full audit.** Every decision, validation verdict, and applied change is recorded before it takes effect, with token usage attached.

4. The two flows

Design-time resolution and run-time repair use the same loop, the same validator and the same write path; only the prompt, the tools exposed, and the trigger differ. In both, the agent finishes by writing configuration, not by asking someone to.

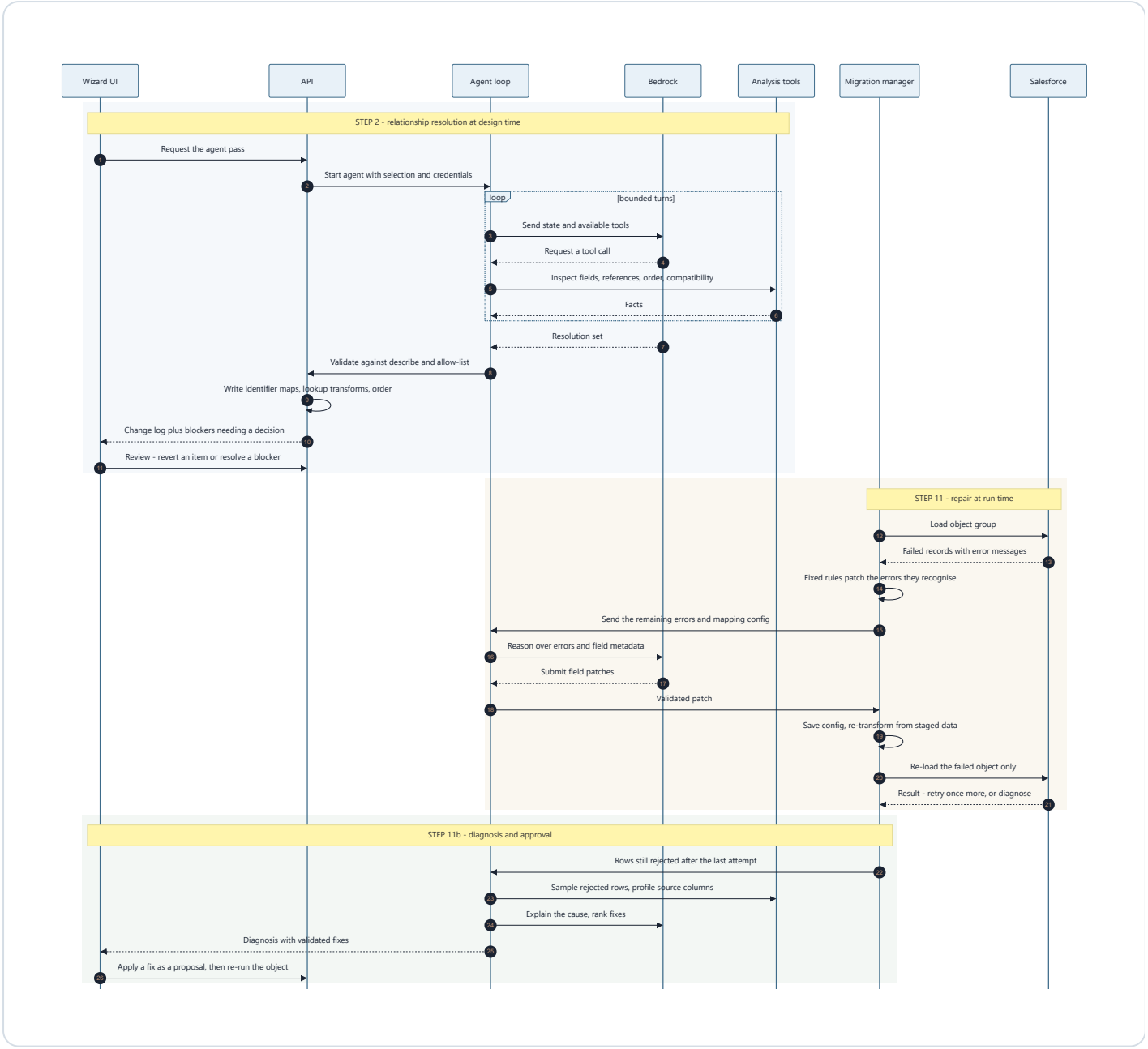


Diagram 2 — Resolution loop at design time; repair loop at run time; diagnosis and human approval for what repair could not fix.

5. Relationship Agent

5.1 The problem it exists to solve

When related records move into Salesforce, Salesforce assigns new identifiers. A child record that pointed at a parent by its source identifier must instead point at the parent's new Salesforce identifier. The pipeline already has the machinery for this — an identifier map populated during load, and a lookup transform that reads it — but somebody has to notice that a given field needs it, register the map under the name the transform will look for, attach the transform to the right field, and ensure the parent loads first.

Today all of that is manual, and it is the single largest cause of failed runs: the symptom appears late — a malformed-identifier error during load, thousands of records in — while the cause is a configuration decision made much earlier. The agent takes over the whole chain, not just the noticing. Detecting the problem and leaving the wiring to a person would keep the expensive half of the work exactly where it is.

5.2 Why the agent generalizes rather than pattern-matching

Identifier remapping is one scenario. It is not the only one, and the important design point is that **we are not building a checklist of known cases**. Deterministic rule checking is exactly where the current implementation already stops: a rule catches the case its author anticipated, and every new client dataset produces relationship shapes nobody anticipated.

The agent is given the whole relational picture rather than a list of tests to run:

- The full source field inventory, including which table every reference field targets.
- The full Salesforce describe for each destination object — lookup targets (including polymorphic targets), required flags, field lengths, and constrained value sets.
- The complete set of objects selected for this migration, and what each one maps to.
- The computed dependency graph, with its edges, cycles, and unresolved parents.
- The naming convention that links an identifier map to the mapping that populates it.

From that surface, the agent reasons about *categories of breakage* — a reference that will not resolve, an ordering that cannot satisfy a dependency, a value that will not survive the destination's constraints — rather than matching a fixed catalogue. When it encounters a relationship shape none of us anticipated, it can still describe the problem in those terms and *fix* it, because the fix is assembled from a small, fixed set of moves that the pipeline already knows how to execute. Novel problem, familiar remedy: that combination is what lets it act on a case nobody wrote a rule for.

5.3 A bounded set of remedies it applies itself

This is what makes autonomous action on open-ended detection safe. Detection is unbounded; action is not. Whatever the agent concludes, the fix it applies must be composed from these primitives, every one of which is something the pipeline already executes today. The agent is choosing and configuring moves, never inventing them.

Remedy	What the agent actually writes
Wire an identifier lookup	Registers the identifier map the parent will populate at load time and attaches the lookup transform, pointed at that map, to the child's reference field
Bring a parent into scope	Adds the parent mapping to the run so its identifiers exist to resolve against
Reorder the load	Writes the parent-first order onto the mappings, as computed by the deterministic resolver
Split into two passes	Configures an insert pass without the reference and a follow-up pass that fills it once identifiers exist — for self-references and cycles
Substitute a constant or default	Sets a fixed target for a required reference whose real target is out of scope
Add a value map	Builds the translation from source vocabulary to the destination's allowed values, read from the describe
Coerce a type	Configures the transform that reformats or bounds the value so the destination accepts it
Raise a blocker	Writes nothing and stops the run with a named remedy — the deliberate choice when no safe automatic fix exists

> **The last row is as important as the others.** An agent that always acts is not trustworthy; one that knows when acting would be a guess is. Refusing to fabricate a parent record or invent a default is treated as a first-class outcome, not a failure to resolve.

5.4 Illustrative scenarios

The following are examples of what this reasoning surface plus this remedy set covers today — deliberately presented as illustrations, not as the definition of the agent's scope.

Scenario	What the agent recognises	What it does about it
Source identifier copied straight into a lookup field	Destination is a lookup; the mapping copies a raw value with no resolution step	Registers the identifier map and attaches the lookup transform to that field
Parent object not selected for migration	A referenced source table has no mapping in the current selection	Pulls the parent into the run and wires the lookup; falls back to a default record where one is safe, and raises a blocker where neither is
Multi-level chain across three or more objects	Transitive dependency edges in the graph	Wires the lookup at every level and lets the resolver order the whole chain
Self-referencing hierarchy, such as a record naming a parent record of the same type	A dependency edge from an object to itself	Configures a two-pass load: insert first, then fill the self-reference
Circular dependency between two objects	A cycle in the graph, which the resolver cannot order	Defers the least critical edge to a second pass so the rest can be ordered, and flags the choice it made
Polymorphic reference with several possible targets	A destination lookup that accepts more than one object type	Configures a multi-target lookup with a defined precedence order
Identifier map referenced by the wrong name	A configured map name that does not match the naming convention	Rewrites it to the canonical name derived from the parent mapping
Junction object joining two others	A source table composed almost entirely of reference fields	Wires resolution on both sides of the join and places it last in the order

5.5 New scenario classes need no code change

The notice channel between the agent and the interface is deliberately open. A notice carries a severity, a category label, the mapping and field it concerns, a plain-language explanation, and either the change that was made or the decision still required. The category label is not a fixed enumeration, so a relationship problem we have never seen is handled correctly without a deployment: it is resolved and logged if it falls inside the remedy set, or it stops the launch and reaches the person who can decide if it does not.

> **Compounding benefit.** Because every notice is audited with its category, the categories that recur across clients become visible. Those graduate into deterministic checks in the compatibility engine — cheaper, instant, and no model call required. The agent effectively acts as a discovery mechanism that keeps hardening the deterministic layer beneath it.

6. Repair Agent

6.1 What it works from

The loader already extracts unique error messages from Salesforce's failed-record output and returns them per object. The repair agent receives that error summary, the object's mapping configuration, and the destination field metadata. Beyond error text and schema it sees at most a couple of short sample values per error class, because the offending value is often the only way to tell a wrong date format from a wrong timezone. Those samples can be switched off entirely.

Salesforce error messages are self-describing enough to reason from, and the same bounded remedy set from the previous section applies. Common classes and their typical resolutions:

Error class	Typical resolution
Value in the wrong format for its type, commonly dates	Reformat with the correct input pattern
Value longer than the destination allows	Bound it to the described length
Value outside a restricted value set	Add a value map built from the destination's own allowed values
Malformed identifier	A missing resolution step — handed back to relationship logic rather than patched locally
Required field empty on some records	Supply a default, or escalate as a blocker if no safe default exists
Duplicate detection, permission and access failures	Not repairable by configuration; surfaced to the user without retry

6.2 Why staging makes repair practical

Because every selected record already sits on our side before any transform runs, a repair is cheap: the failed object is re-transformed from local data and re-loaded on its own. The source system is not re-queried, successfully loaded objects are untouched, and the identifier map entries already written remain

valid. Repair on a streaming pipeline would mean re-extracting and reasoning about partial state; here it is a local, scoped, repeatable operation.

> **Stopping conditions.** A capped number of repair attempts per object per run. Abort immediately if an attempt does not reduce the failure count, if the error class is on the non-repairable list, or if the same patch has already been tried. A run that exhausts repair finishes in the existing completed-with-errors state with the unresolved errors attached — the agent can improve an outcome but never worsen one. A retry that "fixes" a rejection by making the transform discard the row still counts as a failure.

6.3 Fixed rules before the model

Some Salesforce errors have exactly one correct fix, and asking a model for it costs time, tokens and reproducibility. Those are now handled by deterministic rules that run before the Repair Agent, with no model call. When the rules handle every error on an object, the model is not called at all; otherwise it sees only the errors the rules did not recognise.

Error	What the rule wires
A raw source identifier sent to a lookup	The identifier lookup against the map the parent mapping populates
A validation rule stating a minimum and maximum length	A length guard with those limits, tightened a character at a time if the rule rejects its own boundary
Value too long for the field	A length guard at the stated or described length, allowing for line breaks counting double on the wire
"X must be after Y" on dates	A conditional rule that clears the dependent date whenever its prerequisite is empty at the source
A field a validation rule makes required	Maps it from the best-matching source column with a fallback default, or a constant when nothing matches

Every rule-made change is recorded and reversible exactly like an agent change, and appears in the same change log.

6.4 Diagnosis, the assistant, and proposals

Repair deliberately stops at what it can fix safely. When rows are still rejected after the last attempt, the **Diagnosis Agent** takes over. Its job is not to change anything but to explain: it reads a small sample of the rejected rows alongside the staged source rows they came from, sizes the problem by profiling the

relevant source column, and says in plain language whether this is a data problem, a mapping problem or a destination rule. It then ranks two or three fixes with their trade-offs. Where a mapping change would work, the fix carries it, already validated. Where it needs a person — a validation rule to relax, source data to correct — it says who needs to do what.

The migration engineer can then talk to an **assistant** about the session's failures, using the same investigation tools. The assistant cannot change a mapping. When a change is needed it stores a *proposal*: the validated changes, applied to a draft to show a before/after, waiting for a decision. Apply is a separate, deterministic step. It re-validates the change, refuses if the mapping has moved on since the preview or the session is running, records every change so it can be reverted, and then offers to re-run only the affected object.

> **Why this layer asks rather than acts.** By the time a failure reaches diagnosis, the safe mechanical fixes have been tried. What is left usually involves a judgement about the client's data — clear a date, or default it, or leave the record out — which is exactly the decision the first design rules reserve for a person. Making it one click keeps that cheap.

7. State and interfaces

7.1 New state

One new table records agent activity. Every row captures the session and object it concerns, the agent and phase, a digest of the inputs, the change the agent made, the validation verdict, whether it was applied, the prior value it replaced, the model identifier, and token counts. Because the agent writes rather than suggests, this table is doing three jobs at once: it is the audit trail, the mechanism that makes any individual change reversible, and the cost ledger. It also feeds the scenario-frequency analysis described in section 5.5.

Three further tables hold the post-load conversation: the diagnoses written for failing objects, the per-session messages with the assistant, and the proposals awaiting a decision, each with its before/after preview and the diagnosis it came from. An applied proposal writes its changes into the same change log under its own run, so a reviewer can tell a change the agent made on its own from one a person approved.

7.2 Existing state the agents depend on

- **Mapping configurations** — the only design-time write target. The agent adds transform functions, their parameters, and load priorities to the same structure the wizard already saves; nothing structurally new is introduced, which is why its output needs no special handling downstream.
- **Identifier maps** — populated during load, read by lookup transforms. The agent registers them as part of wiring a relationship, always deriving the name from the shared naming helper rather than

composing one. Hand-built names are the most common relationship defect in the current process, and the validator checks for exactly this before anything is written.

- **Job and batch records** — repair retries reuse the same job identifiers so run history stays coherent and a repaired run reads as one run.

7.3 Interfaces

Interface	Purpose
Resolve endpoint	Takes the current selection and credential references. Runs the agent, validates its conclusions, writes the resulting configuration, and returns a change log, the outstanding blockers, and token usage.
Preview mode on the same endpoint	Identical analysis with the write suppressed. Useful for a first run on an unfamiliar client dataset, and for testing.
Revert endpoint	Takes a change reference and restores the prior value from the audit record. Reversal is per item, not per run.
Notices endpoint	Serves current relationship notices for the pre-launch banner and the run view, limited to the current migration attempt so a finding already fixed does not read as still true.
Diagnosis endpoints	List the latest diagnosis per failing object, run one on demand, and turn one of its suggested fixes into a proposal without a model call.
Assistant endpoints	One conversational turn, the conversation history, and apply or reject for a stored proposal.
Resolution history endpoint	Every change an agent applied in the session, grouped by run and tagged with where it came from.

Interface changes in the web application:

- An agent action in Step 4 alongside the existing auto-suggest.
- A change-log panel rather than an approval queue: each item states what was wired, on which field, and why, with a revert control. The default is that the work is done; the user is auditing it, not completing it.
- A separate, prominent list of blockers the agent declined to guess at — these are the only items that actually need a human decision, and they should not be buried among the changes it already handled.
- A notice strip on each mapping card, colour-coded by severity, driven by the open-ended notice categories.

- A repair state on the progress view, so an in-flight repair and its rationale are visible while it happens rather than only in the final error list.
- An assistant panel in the run view: diagnoses with their ranked fixes, the conversation, pending proposals with a before/after, Apply and Reject, and a re-run of the affected object.
- A resolutions drawer listing every applied agent change for the session, with revert.

8. Delivery sequence

#	Stage	Complete when	Estimate
1	Bedrock client and configuration	A round trip to the target model succeeds from the application environment	1–2 days
2	In-memory configuration refactor	Dependency resolution and compatibility checks run on draft configurations without database writes	2–3 days
3	Tool adapters	Each tool is independently exercisable against real source and destination credentials	2–3 days
4	Agent loop and validator	A deliberately malformed conclusion is rejected rather than written	3–4 days
5	Relationship agent in preview mode	The change log for a known multi-object set matches what an experienced engineer would wire by hand	4–6 days
6	Audit trail and revert	Every change is recorded with its prior value and can be reversed individually	2 days
7	Enable writes, plus the wizard change-log view	A run wired entirely by the agent completes without hand-editing a mapping	3–5 days
8	Repair agent	A seeded format failure self-heals on the first retry	3–5 days
9	Repair visibility in the run view	Repair state and rationale are visible live	1–2 days

What the estimates assume

End to end this is **roughly four to five weeks** for one engineer already familiar with this codebase, working the stages in order. Two intermediate points are worth planning against rather than the end date alone: the agent can be evaluated against real client data at the end of stage 5, around **two and a half to**

three weeks, and it is permitted to write configuration at the end of stage 7, around **three and a half to four weeks**. Repair lands last.

- **Sequential, because each stage depends on the one before it.** The exception is stage 9, which is presentation only and can trail the rest or be dropped from the first release without affecting behaviour.
- **Stage 5 carries the widest range because it is the only empirical stage.** Everything else is ordinary engineering with a testable finish line; prompt and tool-surface iteration converges when it converges. It is also the stage where the range narrows fastest once work starts, so it is the right place to re-forecast from.
- **Stages 1 to 7 carry no runtime risk and can absorb slippage.** None of them execute during a migration, so they can be delivered around live client work rather than blocking it.
- **Excluded: waiting for Bedrock access.** Stage 1 measures the build once credentials exist, not the approval that precedes it. Nothing can start without that approval, which makes it the critical path ahead of any of these numbers.
- **Excluded: review cycles that depend on other people.** Stage 5 is judged by comparing the agent's change log against hand-wiring on a real dataset, so it assumes an experienced engineer is available to do that comparison.

> **Autonomy is switched on at stage 7, not stage 5.** The agent runs in preview for two stages first, which means we can compare its change log against hand-wiring on real client data before it is permitted to write anything, and the revert path exists before the first write does. Stages 1 to 7 also carry no runtime risk: none of them execute during a migration. Repair is the only component that runs inside a live migration, it arrives last, and it sits behind both a feature flag and its own attempt cap.

> **Delivered beyond this plan.** All nine stages are complete. Since then, the repair stage has gained fixed rules that run ahead of the model (6.3), a Diagnosis Agent for failures repair cannot fix, and an assistant whose fixes are applied only on approval (6.4). Alongside these: a preview-first way to fill a single column on records that have already migrated, and field-matching confidence scored on meaning rather than name resemblance.

9. Operational posture

Concern	Handling
Bedrock unavailable or throttled	Adaptive retry, then fall through to the deterministic path; the run proceeds as it does today
What the model receives	Schema and error text — field names, types, allowed values, and Salesforce error messages — plus, for repair and diagnosis only, a capped and truncated sample of rejected rows and source-column profiles. One setting removes those values. No extract dumps, no credentials, no attachments
Fixes that need judgement	Diagnosed and offered as proposals with a before/after. Nothing is applied without a person, applying never calls the model, and a proposal made stale by a later edit is refused rather than applied over it
Credential handling	Resolved server-side by reference; never declared in a tool schema, never placed in a prompt
Cost	A resolution pass is a modest number of tokens against a schema, not against data; a repair is smaller still, and none at all when the fixed rules handle every error. All are capped and logged per call, so spend is measurable per migration from day one
Latency	Resolution is user-initiated at design time and takes tens of seconds. Repair adds one model call and a scoped re-transform to a run that had already failed
Non-determinism	Deterministic decoding settings, and every conclusion validated against live schema before it is written. Two runs may reach the same outcome by different routes; neither can write an invalid configuration
Wrong-but-valid changes	The residual risk of autonomy: a change that passes validation but is not what the client wanted. Contained by the change log, per-item revert, and the pre-flight gate at step 4 — not by asking a human to approve every item, which is the review nobody performs carefully
Auditability	Every decision, verdict and applied change is attributable to a session, an object, and a model version, with the value it replaced

---

Staged pipeline: extract all selected data to our side, then transform, load, and record identifier mappings. Agents resolve relationships and write the configuration themselves; the deterministic pipeline executes it.